

מערכות הפעלה – מטלה 2

ת"ז – 212214316 , 211544960

TQ1

1. נשים לב כי בכל דרך שנבחר יהיו לנו $2500 = 200 + 2000 + 50 + 250$ חזרות.
בנוסף לכך – בדרך $\text{int } 0x80$ יהיו לנו $300 = 220 + 80$ חזרות נוספות לכן סך החזרות בדרך זו יהיו 2,800.
כמו כן – בדרך syscall המודרני יהיו לנו $100 = 70 + 30$ חזרות נוספות לכן סך כל החזרות בדרך זו יהיו 2,600.
2. מהנתון התוכנה תפעל ב-10,000 קריאות. לפי החישוב שלנו מסעיף א' – כל קריאה תצרוך 2,600 או 2,800 חזרות לפי סוג המסלול – $\text{int } 0x80$ או syscall המודרני בהתאמה. מכך, סך כל הקריאות לפעולה יהיו 28,000,000 ו-26,000,000 בהתאמה לפי סוג המסלול. לכן – ההפרש בין הקריאות כלומר החיסכון בשימוש ב syscall המודרני על פני $\text{int } 0x80$ יהיה 2,000,000 קריאות.
3. נראה מהטבלה כי עבודת ה kernel האמיתית היא 2,000 חזרות, ולכן נראה כי בדרך ה- $\text{int } 0x80$ ה-overhead יהיה $2800 - 2000 = 800$ חזרות, בעוד בדרך syscall המודרני ה-overhead יהיה $2600 - 2000 = 600$ חזרות.
מכך – אחוז ה-overhead עבור $\text{int } 0x80$
$$\frac{2800-2000}{2800} \cdot 100 = \frac{800}{2800} \cdot 100 = 28.6\%$$

עבור syscall המודרני
$$\frac{2600-2000}{2600} \cdot 100 = \frac{600}{2600} \cdot 100 = 23.1\%$$
4. נחשב – $2800 \cdot N = 2600 \cdot N + 10000$
$$200N = 10000 \rightarrow N = 50$$

לכן – המספר המינימלי N עבורו הקריאה ל syscall המודרני תהיה 'זולה' יותר מ- $\text{int } 0x80$ תהיה עבור $N=51$.

TQ2

1. E1 – יש שימוש ב-System Call, מכיוון שב-user mode אנחנו לא יכולים לכתוב באופן ישיר לחומרה, אנחנו מבקשים מה- kernel לבצע פעולת I/O.
E2 – Software Interrupt. פעולת ה- int הינה פעולת interrupt והמספר 3 מצביע על וקטור הכניסה השלישי ב-CPU ב-IDT, לכן המתכנת קורא באופן מפורש באמצעות הפקודה לעצור את ה-CPU ולקפוץ ל- kernel באמצעות debugger.
E3 – Exception. נשים לב כי פעולת חילוק ב-0 איננה חוקית ותיצור בעיה ב-CPU לכן מדובר ב-Exception.
E4 – Signal Masking – המשתמש לא יכול לשכתב זיכרון ב- kernel באופן ישיר, לכן קורא ל-signal mask כדי לעדכן את ה-thread.
E5 – Signal Wait – המשתמש לא יכול לשלוט ב-scheduler של ה- kernel , לכן הוא מבצע פעולת signal wait שאומרת ל- kernel להוריד אותו בעדיפות בתור.

2. עבור E1 :

ה-CPU אכן נכנס ל-kernel mode כיוון שאנחנו לא יכולים לכתוב לחומרה באופן ישיר (הסיבה העיקרית לכך).

אין context switch ל-process / thread אחר – ה-kernel מבצע את פעולת ה-write בזמן שה-CPU משנה את ה-privileged levels, אבל זה לא משנה את ה-context של ה-thread.

עבור E2 :

אנחנו נכנסים ל-kernel mode מכיוון שהופעלה פעולת interrupt שמפעילה את ה-IDT באופן אוטומטי, ה-CPU נכנס ל-kernel mode (הסיבה העיקרית לכך).

אין לנו בהכרח context switch – ה-CPU קופץ ישירות ל-breakpoint handler של ה-kernel בתוך מסגרת ה-context שה-thread מריץ.

עבור E3 :

אנחנו נכנסים ל-kernel mode מכיוון שכאשר שהחומרה מזהה פעולה אריתמטית שגויה, ה-CPU מכין exception באופן אוטומטי שמכניס אותו ל-kernel mode.

אין לנו context switch אבל בדרך כלל הפעולה תוביל לחיסול ה-process.

הסיבה העיקרית שבגללה אנחנו נכנסים ל-kernel היא טיפול בשגיאת חומרה בזמן ריצה.

3. ה-thread שמקבל ויטפל בו הוא T2.

נשים לב כי signal שנשלח מבחוץ מגיע אל ה-process כולו ולא אל thread ספציפי, לכן נוחת בתור ה-signals המשותפים של ה-process.

בשורה T2 – sigmask ביצע חסימה של ה-signal, פעולה זו מונעת ממנו להפריע לו באופן אסינכרוני. כעת נראה כי אם signal ממתין בתור כלשהו בתור של ה-process וישנו thread שמבצע אליו קריאה סינכרונית (לדוגמה sigwait), ה-kernel מחויב לנתב את ה-signal ישירות אליו, לכן ה-kernel מעיר את T2, מעביר לו את ה-signal וממשיך בהרצת T2 באופן רגיל. נראה כי מכיוון ש-T2 צריך את ה-signal, ה-signal נעלם ולכן T1 לא מקבל את ה-signal.

4. i – לא נכון. תיקון – A System call is a privileged switch from user mode to kernel mode in the same process/thread context

ii – לא נכון. תיקון – int 3 is a software interrupt because it uses the int instruction (invokes a low-level handler from the IDT)

iii – לא נכון. תיקון – The signal handler can run on any thread that does not have that signal blocked

iv – נכון.

v – נכון.

TQ3

1. הפעולות יפעלו באופן הבא :

#	Order of critical sections	Value printed by B	Final value of counter
1	A -> B -> H	1	0
2	A -> H -> B	0	0

3	B -> A -> H	0	0
4	B -> H -> A	0	0
5	H -> A -> B	0	0
6	H -> B -> A	-1	0

2. התוכנה יכולה להיכנס ל-deadlock גם ב-thread A וגם ב-thread B, כאשר מתקבל ה-signal מ-SIGUSR1.

על מנת להגיע ל-deadlock בזמן קבלת ה-signal, אחד מ-thread A או thread B נמצאים בתוך ה-critical section שלהם, כלומר אחרי ביצוע פעולת pthread_mutex_lock(&lock) ולפני ביצוע פעולת pthread_mutex_unlock(&lock). נראה כי ניכנס ל-deadlock כאשר thread A / B כבר ביצעו lock, ולכן כאשר H יקרא ל-lock, נמתין שה-mutex ישתחרר אך אין פעולה שתשחרר אותו, לכן אנחנו ניכנס ל-deadlock.

3. i – צריך לחסום זמנית את SIGUSR1 ב-thread A + B, כיוון ששניהם נכנסים ל-critical section.

ii – צריך לחסום את SIGUSR1 בסימונים A1 + B1, כיוון שנדרש לחסום אותו לפני הכניסה ל-critical section.

iii – צריך לשחזר את ה-signal mask הקודם בסימונים A2 + B2, כיוון שנדרש לשחזר אותו אחרי היציאה מה-critical section.

iv – נשים לב כי מצב זה ימנע deadlock מכיוון שהחסימה ב-A1 / B1 תגרום ל-signal של SIGUSR1 להיות ב-pending לאחר הסימונים A2 / B2 (שם השתמשנו ב-mask), ובכך יתאפשר כי ב-critical section של thread A / B יבוצעו lock ו-unlock ללא הפרעה, ולכן יימנע deadlock.

v – לא תהיה חסימה של SIGUSR1 בשום מצב מכיוון שכאשר ה-thread יגיע ל-A2 / B2, יבוצע שחזור של ה-signal mask, כלומר ביטול חסימה של SIGUSR1. כלומר, אם ה-signal יגיע בתוך ה-critical section הוא יבוצע מיד לאחר ה-critical section, ואם הוא קורה מחוץ ל-critical section הוא יבוצע כרגיל.

TQ4

1. הבאג בפונקציה switch_to הוא שהפונקציה מעדכנת את current_uthread לאחר הקריאה ל-siglongjmp ומכך לא יתבצע עדכון ל-current_uthread. ביצוע siglongjmp בתוך switch_to גורם ל-CPU להחליף context באמצע ריצת switch_to, ולכן מדלג על המשך השורות בפונקציה ולא מבצע אותן אף פעם. כמו כן, הבאג גורם ל-yield לשמור את ה-context הלא נכון כיוון שאנחנו לא מעדכנים את ה-current_uthread – הוא יישאר 1- כאשר נגיע ל-yield, ננסה לבצע את השורה של sigsetjmp(env[me],1) – כלומר sigsetjmp(env[-1],1) – ולכן נקבל אינדקס 1- למערך ומכך נקבל שגיאה.

2. הדרך לתקן את הבאג היא להחליף את סדר השורות בתוך הפעולה – קודם נבצע את השורה `current_uthread = tid` ולאחר מכן נבצע את השורה `siglongjmp(env[tid],1)`.
החלפת השורות תוודא את ביצוע השמירה של `current_uthread`.
3. הפלט של הפעולה כאשר `main` קורא ל-`switch_to(0)` יוצג בסדר הבא :

I'm here

I'm there

I'm here again

I'm there again

4. נתייחס לשני המקרים :

`i - 0 = sigsetjmp(env[me],1)` – מצב זה יקרה כאשר הפונקציה נקראת באופן ישיר ע"י ה-`thread` שרץ באותו הרגע, כלומר `thread me`. ה-`context` נשמר ולכן בגלל כך הפעולה `sigsetjmp` תחזיר 0. ההשוואה תהיה נכונה ומכך נעבור ל-`switch_to()`.

`i - 0 ≠ sigsetjmp(env[me],1)` – מצב זה יקרה כאשר ה-`thread` השני קורא ל-`siglongjmp(env[me],1)` ממקום אחר. לכן, ה-CPU ישחזר את המצב מה-`sigsetjmp`, ויחזור לרגע אחרי ביצוע `sigsetjmp`. נשים לב כי מכיוון שהתבצעה קפיצה, הפונקציה תחזיר ערך 1, ההשוואה של `sigsetjmp` ל-0 תהיה לא נכונה לכן נדלג על בלוק הפעולה שמתחתיו, יסיים את פעולת ה-`yield` וימשיך כרגיל.

5. הרצת הפעולה על שני `cores` לא תשנה את המהירות, מכיוון ששני ה-`threads` הם `uthread` שרצים על `krenel` אחד, וזמן הריצה ע"י שני `cores` לא ישפר זמן ריצה של `kernel thread` אחד.